Theoretical Chemistry and Computational Modelling

Programming and Computational Techniques

Python programming • Unit 1: the fundamentals

David Ferro Costas

Academic year 2026/2027

Outline

Contents

1. Python as a scientific calculator
2. Variables, data types and input/output
3. Control flow: conditionals, loops and exceptions
4. Sequences: lists, tuples and strings
5. Dictionaries
6. Reading and writing files
7. Wrap-up and practice

By the end of this unit you should be able to...

- evaluate formulas with Python and the `math` module
- read data, convert types and print formatted results
- control the flow of a program with `if`, `while` and `for`, using indented blocks
- store and manipulate data in lists, tuples, strings and dictionaries
- read and write text files
- read error messages, handle exceptions and avoid common pitfalls

1 Python as a scientific calculator

Arithmetic operators

Op.	Meaning	Example	Result
`+`	addition	`7 + 2`	`9`
`-`	subtraction	`7 - 2`	`5`
`*`	multiplication	`7 * 2`	`14`
`/`	division	`7 / 2`	`3.5`
`//`	floor division	`7 // 2`	`3`
`%`	remainder (modulo)	`7 % 2`	`1`
`**`	power	`7 ** 2`	`49`

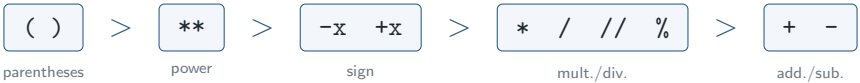
Python 3. `/` *always* returns a `float`, even if the division is exact: `6 / 3` gives `2.0` (in Python 2, `7 / 2` gave `3`).

Watch out! `//` rounds *down* (towards $-\infty$), not towards zero:

```
>>> -7 // 2
-4
```

Tip. `n % 2 == 0` tests whether `n` is even.

Operator precedence



Same precedence? Evaluate from left to right

```
>>> 10 / 2 * 5
25.0
>>> (10 / 2) * 5
25.0
>>> 10 / (2 * 5)
1.0
```

Exception: `**` is evaluated from right to left

```
>>> 2 ** 2 ** 3
256
>>> (2 ** 2) ** 3
64
```

Watch out! `-2 ** 2` gives `-4`: the power is computed before the sign. When in doubt, use parentheses.

The `math` module

docs ↗

Mathematical functions and constants live in a *module* that must be imported:

```
>>> import math
>>> math.sqrt(2)
1.4142135623730951
>>> math.exp(1)
2.718281828459045
>>> math.log(100)        # natural log!
4.605170185988092
>>> math.log10(100)
2.0
>>> math.sin(math.pi / 2)    # radians!
1.0
>>> math.degrees(math.pi)
180.0
```

Three ways to import

```
import math                 # math.sqrt(2)
from math import pi, sqrt   # sqrt(2)
import math as m            # m.sqrt(2)
```

Avoid `from math import *`: you lose track of where each name comes from, and it may overwrite your own variables.

Most used `pi`, `e`, `inf`, `sqrt`, `exp`, `log`, `log10`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan2`, `degrees`, `radians`, `factorial`, `isclose`

Floating-point, integer and complex numbers

EXTRA

Floats are stored with 53 binary digits (∼16 decimal digits), so most decimals are only approximated: *round-off errors*

```
>>> 6.022e23            # 6.022 x 10^23
6.022e+23
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
```

Watch out! Never compare floats with ==. Use `math.isclose(a, b)` or `abs(a - b) < tol`.

Integers have arbitrary precision

```
>>> 2 ** 100
1267650600228229401496703205376
```

Complex numbers are built in (`j` is the imaginary unit), handy in quantum mechanics

```
>>> z = 3 + 4j
>>> abs(z)
5.0
>>> z.conjugate()
(3-4j)
>>> z * (1 - 2j)
(11-2j)
```

Complex functions are in the `cmath` module (`cmath.exp`, `cmath.sqrt`, ...)

2 Variables, data types and input/output

Variables and assignment

- Variables are *names for values*; = **assigns** a value to a name (it is not an equality)
- A variable is created when you first assign it; using it before gives a `NameError`
- **Dynamic typing**: no declarations. The type belongs to the value, so a name can later refer to a value of another type
- Everything is an **object**: a piece of memory with a value, a type and associated operations (*methods*)

```
n_atoms = 3
energy = -76.0268       # hartree
molecule = "H2O"
```

The type follows the value

```
>>> a = 10
>>> type(a)
<class 'int'>
>>> a = 10.          # or float(10)
>>> type(a)
<class 'float'>
>>> a + 1            # int + float
11.0
>>> a = "ten"        # now a str
>>> type(a)
<class 'str'>
```

Naming variables

Rules

- letters, digits and underscore (_) only
- cannot start with a digit:
  `77david = "name"` → `SyntaxError`
- case sensitive: `a` and `A` are different
- cannot be a keyword (right)

Name		
`n_electrons`	✓	
`theta2`	✓	
`2theta`	×	starts with a digit
`total-energy`	×	- is a minus sign
`lambda`	×	keyword

The 35 keywords

```
False   await     else     import   pass
None    break     except   in       raise
True    class     finally  is       return
and     continue  for      lambda   try
as      def       from     nonlocal while
assert  del       global   not      with
async   elif      if       or       yield
```

Style (PEP 8).
`lower_case_with_underscores` for variables and functions, `CapWords` for classes (later). Meaningful names: `area = pi * radius**2`, not `x = y * z**2`. Do not reuse built-in names (`list`, `sum`, `max`, `input`).

Reading error messages

Python stops at the first error and tells you *where* it happened and *what* went wrong:

names.py

```
a = 10
print(A)
```

```
$ python3 names.py
Traceback (most recent call last):
  File "names.py", line 2, in <module>
    print(A)
          ^
NameError: name 'A' is not defined. Did you mean: 'a'?
```

- **Where**: file, line number and the offending code
- **What**: the last line, with the type of error and a message

Tip. Read error messages from the bottom up. Later we will learn to *catch* them with `try`/`except`.

Error	Typical cause
`SyntaxError`	invalid code, e.g. missing `:`
`NameError`	typo, wrong case, undefined
`TypeError`	wrong type: `"1" + 4`
`ValueError`	bad value: `int("3.5")`
`IndentationError`	misaligned block
`ZeroDivisionError`	`1 / 0`
`IndexError`	index out of range
`KeyError`	key not in a dictionary
`FileNotFoundError`	the file does not exist

Updating variables

Augmented assignment

Long form		Short form
`x = x + 1`	⇔	`x += 1`
`x = x - 1`	⇔	`x -= 1`
`x = x * 2`	⇔	`x *= 2`
`x = x / 3`	⇔	`x /= 3`
`x = x // 3`	⇔	`x //= 3`
`x = x ** 2`	⇔	`x **= 2`

Typical use: counters (`count += 1`) and accumulators (`total += value`)

Multiple assignment

```
x, y, z = 0.0, 0.757, -0.469
print(x, y, z)
```

```
0.0 0.757 -0.469
```

Swapping two values needs no auxiliary variable:

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

```
2 1
```

Python 3. `x /= 3` always produces a `float`, even if `x` was an integer.

Basic data types

Type	Meaning	Examples
`int`	integer	42, -7
`float`	real number	3.14, 1.6e-19
`complex`	complex number	2 + 1j
`str`	text (string)	`"H2O"`
`bool`	logical value	`True`, `False`
`NoneType`	"nothing"	`None`

`type()` tells you the type of any object:

```
>>> type(6.022e23)
<class 'float'>
>>> type("6.022e23")
<class 'str'>
```

Also useful: `abs(-2.7)` → `2.7`

Type conversion

```
>>> int("42")
42
>>> float("1e-3")
0.001
>>> int(3.99)           # truncates
3
>>> round(3.14159, 2)
3.14
>>> int("3.5")
ValueError: invalid literal ...
```

Watch out! `round(2.5)` gives 2 and `round(3.5)` gives 4: ties go to the even number.

Strings

- Text between quotes: `'single'` or `"double"` (equivalent: pick one and stick to it)
- Triple quotes (`"""..."""`) for multi-line text
- Special characters: `"\n"` (new line), `"\t"` (tab)
- `+` concatenates, `*` repeats, `len()` gives the length

```
>>> print(3 * "SCF...\n" + "Done!")
SCF...
SCF...
SCF...
Done!
```

Numbers inside strings are just text

```
>>> "1" + "4"
'14'
>>> 1 + 4
5
>>> "1" + 4
TypeError: can only concatenate str ...
>>> "1" + str(4)
'14'
>>> int("1") + 4
5
```

Strings are also *sequences* of characters: indexing, slicing and string methods come later, with lists.

Booleans and comparisons

Two values: `True` and `False` (capitalised!)

Comparison operators give a boolean

`==`	equal	`3 == 4` → `False`
`!=`	not equal	`3 != 4` → `True`
`<, >`	less, greater	`3 > 4` → `False`
`<=, >=`	... or equal	`3 <= 4` → `True`

Watch out! = assigns a value; == compares two values.

Priority: comparisons > `not` > `and` > `or`.
The last example on the right means
`(not (3 == 4)) and (1 != 2)`

Logical operators: `not`, `and`, `or`

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
>>> 3 == 4 and 1 != 2
False
>>> 3 == 4 or 1 != 2
True
>>> not 3 == 4 and 1 != 2
True
```

More on booleans

EXTRA

Chained comparisons

Python allows chaining comparison operators:

```
1 < x and x < 10   ⇔   1 < x < 10
```

A puzzle

```
>>> 1 < 3 < 0 == True
False
```

== is also a comparison operator, so this is a chain:

```
1 < 3 and 3 < 0 and 0 == True
```

and `3 < 0` is `False`. Use parentheses to make your intention explicit.

"Truthiness"

Any object can be used as a condition. Zero and empty objects count as `False`; everything else as `True`:

```
>>> bool(0), bool(""), bool([])
(False, False, False)
>>> bool(-1), bool("False")
(True, True)
```

`bool` is a subtype of `int`:

```
>>> True == 1
True
>>> True + True
2
```

Output: `print()`

`print()` writes to the **standard output** (the screen); arguments are separated by a space, and a new line is added at the end:

```
from math import pi
radius = 5.3                    # m
area = pi * radius**2           # m^2
print("radius =", radius, "m, area =",
      round(area, 2), "m^2")
print(area)
```

```
radius = 5.3 m, area = 88.25 m^2
88.24733763933729
```

Python 3. `print` is a function: Python 2's `print "E =", E` is a syntax error.

Optional arguments `sep` and `end`

```
print("H", "O", "H", sep="-")
print("Fe", end="")
print("2+")
```

```
H-O-H
Fe2+
```

Tip. `print()` shows text *without* quotes; an interactive session shows the value's representation:

```
>>> print("H2O")
H2O
>>> "H2O"
'H2O'
```

Formatted output: f-strings

docs ↗

`print` alone does not let you choose, e.g., the number of decimals. **f-strings** (Python $\geq$ 3.6) build a formatted string: put `f` before the quotes; any *expression* between braces is evaluated and inserted, with an optional format after a colon:

```
mol = "H2O"
n_atoms = 3
energy = -76.0267656     # hartree
print(f"E({mol}) = {energy:.4f} Eh")
print(f"3N = {3 * n_atoms}")
```

```
E(H2O) = -76.0268 Eh
3N = 9
```

Tip. `f"{x = }"` prints the expression and its value: very useful for debugging.

Anatomy of a format



Type

d	integer
f	decimals
e	scientific
g	f or e
s	string

Alignment

<	left (default for text)
>	right (default for numbers)
^	centred

All parts are optional: `{x}`, `{x:.2f}`, `{x:>8}`, ...

Format specification: examples

Spec	Meaning	Result
`x = -76.0267656`		
`.4f`	4 decimals	`-76.0268`
`12.6f`	width 12, 6 decimals	`␣␣-76.026766`
`.3e`	scientific notation	`-7.603e+01`
`.4g`	4 significant digits	`-76.03`
`n = 7`		
`5d`	integer, width 5	`␣␣␣␣7`
`03d`	zero-padded, width 3	`007`
`s = "H2O"`		
`>8`	right-aligned, width 8	`␣␣␣␣␣H2O`
`<8`	left-aligned, width 8	`H2O␣␣␣␣␣`
`^8`	centred, width 8	`␣␣H2O␣␣␣`

␣ = space

Printing a table

Fixed widths keep the columns aligned:

```
data = [("H", 1, 1.008),
        ("C", 6, 12.011),
        ("Cl", 17, 35.45)]
print(f"{'El':<4}{'Z':>4}{'Mass':>10}")
for sym, Z, m in data:
    print(f"{sym:<4}{Z:>4d}{m:>10.3f}")
```

```
El      Z      Mass
H       1     1.008
C       6    12.011
Cl     17    35.450
```

Tip. The same formats work when writing files, later in this unit.

Older ways to format: `format()` and %

EXTRA

You will find two older styles in existing code. They use the same format specifications:

```
mol, E = "H2O", -76.0267656
print(f"E({mol}) = {E:.4f} Eh")              # f-string (recommended)
print("E({}) = {:.4f} Eh".format(mol, E))    # str.format() method
print("E(%s) = %.4f Eh" % (mol, E))          # % operator (oldest)
```

```
E(H2O) = -76.0268 Eh
E(H2O) = -76.0268 Eh
E(H2O) = -76.0268 Eh
```

`str.format()`

- `{}` are filled with the arguments, in order
- or by position: `"{1} {0}".format("A", "B")` gives `'B A'`
- useful when the template is written before the values are known

The % operator

- placeholders `%s`, `%d`, `%8.3f`, `%.2e`, ...
- values after `%`, in a tuple if there are several
- comes from C (`printf`); still common in older scientific code

Input: `input()`

`input()` reads from the **standard input** (the keyboard): it shows a message, waits for the user and returns what was typed *as a string*:

```
from math import pi

radius = input("Radius (m): ")  # str
radius = float(radius)          # float
area = pi * radius**2
print(f"Area = {area:.2f} m^2")
```

```
Radius (m): 5.3
Area = 88.25 m^2
```

Both steps are usually written in one line:

```
radius = float(input("Radius (m): "))
```

Watch out! Forgetting the conversion does not always raise an error...

```
>>> radius = "5.3"
>>> radius * 2
'5.35.3'
>>> radius ** 2
TypeError: unsupported operand ...
```

Python 3. `input()` always returns a string. Python 2's `input()` evaluated the text and `raw_input()` returned a string; `raw_input` no longer exists.

Putting it together: the Arrhenius equation

$$k = A\,e^{-E_a/RT}$$

```
import math

A = 1.0e13        # prefactor (1/s)
Ea = 50.0e3       # activation energy (J/mol)
R = 8.314462618   # gas constant (J/(mol K))

T = float(input("Temperature (K): "))
k = A * math.exp(-Ea / (R * T))
print(f"k({T:.2f} K) = {k:.3e} s^-1")
```

```
Temperature (K): 298.15
k(298.15 K) = 1.739e+04 s^-1
```

Good habits already visible

- one quantity per variable, with a meaningful name
- units written in comments
- constants defined once, at the top
- input converted to `float` immediately
- result printed with sensible precision

Try it. How much faster is the reaction at 373.15 K?

3 Control flow: conditionals, loops and exceptions

Code blocks and indentation

From now on, code is organised in **blocks**: groups of lines that run together, repeatedly, or not at all. Each language marks blocks in its own way:

Fortran

```
if (x < 12) then
   print *, x
end if
```

C

```
if (x < 12) {
    printf("%d", x);
}
```

Python

```
if x < 12:
    print(x)
```

In Fortran and C indentation is only cosmetic. In Python **indentation is the syntax**: no `end if`, no braces.

The rules

- A line ending in a colon `:` (`if`, `else`, `for`, `while`, ...) opens a block
- All the lines of the block are indented by the same amount: **4 spaces** (PEP 8)
- The block ends at the first line that goes back to the previous level
- Blocks can be nested: each level adds 4 more spaces
- A block cannot be empty: write `pass` if there is nothing to do (yet)

Nested blocks, and why indentation matters

Nested blocks

```
if T > 273.15:                level 0
    if T < 373.15:            level 1
        state = "liquid"      level 2
    else:                     level 1
        state = "gas"         level 2
else:                         level 0
    state = "solid"           level 1
print(state)                  level 0
```

With `T = 300.0` this prints `liquid`. Each `else` belongs to the `if` at its *own* level.

Indentation changes the meaning

Same lines, different indentation, different program:

```
x = 5
if x > 10:
    print("large")
    print("checked")
```

Prints nothing: both `print` lines are inside the block.

```
x = 5
if x > 10:
    print("large")
print("checked")
```

```
checked
```

Watch out! A wrongly indented line often gives no error at all: just a different (wrong) result.

Indentation errors

When the indentation does not make sense, Python refuses to run the program:

Problem	Code	Message
no indentation after the colon	`if x < 12:` `print(x)`	`IndentationError: expected an indented block after 'if' statement on line 1`
indented line, but no block was opened	`x = 5` `    print(x)`	`IndentationError: unexpected indent`

going back to a level that does not exist	`if x < 12:`
	`    print(x)`
	`  print("done")`
		`IndentationError: unindent does not match any outer indentation level`

tabs and spaces mixed in the same block	`if x < 12:`
	`    print(x)`
	`TAB print("done")`
		`TabError: inconsistent use of tabs and spaces in indentation`

Tip. Set your editor so that the Tab key inserts 4 spaces (most Python editors already do) and shows indentation guides.

Conditionals: `if` / `elif` / `else`

docs ↗

Conditionals choose which block runs, depending on one or more conditions:

```
x = int(input("Natural number: "))
if x <= 0:
    print("Not a natural number")
    print("Same indentation, same block!")
elif x <= 10:
    print("The number is in [1, 10]")
elif x <= 100:
    print("The number is in [11, 100]")
else:
    print("The number is greater than 100")
print("This line always runs")
```

```
Natural number: 42
The number is in [11, 100]
This line always runs
```

- Each condition ends with a colon `:`
- The body of each branch is an indented block
- Conditions are checked from top to bottom; *only the first* true branch runs, so the order matters
- `elif` and `else` are optional

> **Tip.** To compare one value against many options, see `match` (two slides ahead).

Conditionals: more examples

```
pH = float(input("pH: "))
if pH < 7:
    label = "acidic"
elif pH > 7:
    label = "basic"
else:
    label = "neutral"
print(f"pH {pH} -> {label}")
```

```
pH: 3.2
pH 3.2 -> acidic
```

Conditions can be combined

```
if 273.15 < T < 373.15:
    print("liquid water (1 atm)")
```

Conditional expression (one-line `if`/`else`)

```
parity = "even" if n % 2 == 0 else "odd"
```

Common errors

- `if x > 3` → missing colon: `SyntaxError`
- `if x = 3:` → use `==` to compare (`=` assigns)
- `if x < 0 or > 10:` → each side needs a full comparison: `x < 0 or x > 10`
- `if pH == 7.0:` → exact equality of floats is rarely what you want
- a line with the wrong indentation: `IndentationError`, or a different program

Pattern matching: `match` (Python $\geq$ 3.10)

docs ↗

Compares one value against a series of *patterns*, like `select case` in Fortran or `switch` in C:

```
element = "Si"
match element:
    case "H":
        valence = 1
    case "O":
        valence = 2
    case "C" | "Si":
        valence = 4
    case _:
        valence = None
print(element, valence)
```

```
Si 4
```

- Only the *first* matching case runs
- `|` means "or" between patterns
- `case _:` matches anything, like `else`

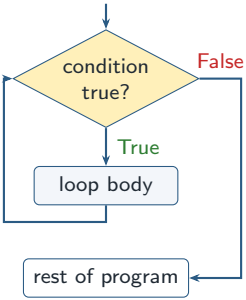
The same with `if/elif`

```
if element == "H":
    valence = 1
elif element == "O":
    valence = 2
elif element in ("C", "Si"):
    valence = 4
else:
    valence = None
```

`match` can do much more (e.g. match the structure of lists or dictionaries), beyond this unit.

`while` loops

docs ↗



Watch out! If the condition never becomes `False` you get an *infinite loop*. Stop it with `Ctrl+C`.

```
while condition:
    statement(s)
```

- Repeats the indented block *while* the condition is `True`; the number of iterations is not known in advance
- Something inside the loop must eventually make the condition `False`

Example: the first power of 2 above 1000

```
n = 1
while n < 1000:
    n *= 2
print(n)
```

```
1024
```

`break` and `continue`

- `break`: leave the loop immediately (essential in a `while True:` loop)
- `continue`: skip the rest of the block and jump to the next iteration
- Both work in `while` and `for` loops

Example: an iterative solver
Newton's method for $x^2 = 2$:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{2}{x_n}\right)$$

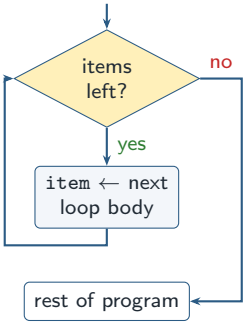
Iterate until the change is below a threshold: the same structure as an SCF cycle or a geometry optimisation (where a maximum number of iterations is also set).

```
x = 1.0             # initial guess
tol = 1e-10         # threshold
iteration = 0
while True:
    iteration += 1
    x_new = 0.5 * (x + 2.0 / x)
    if abs(x_new - x) < tol:
        break       # converged
    x = x_new
print(iteration, "iterations")
print(f"x = {x_new:.12f}")
```

```
5 iterations
x = 1.414213562373
```

`for` loops

docs ↗




```
for char in "NaCl":
    print(char, end=" ")
```

```
N a C l
```

```
for item in sequence:
    statement(s)
```

- Visits the elements of a sequence one by one: the number of iterations is known
- Works with strings, lists, tuples, dictionaries, `range()`, files, ...
- `break` and `continue` work as in `while` loops

```
for atom in ["C", "H", "H", "O"]:
    print(atom, end=" ")
```

```
C H H O
```

Counting with `range()`

`range(stop)`	$0, 1, \ldots, \text{stop} - 1$
`range(start, stop)`	$\text{start}, \ldots, \text{stop} - 1$
`range(start, stop, step)`	$\text{start}, \text{start} + \text{step}, \ldots$

Watch out! Python starts counting at 0 and `stop` is *never* included.

```
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> list(range(1, 6))
[1, 2, 3, 4, 5]
>>> list(range(2, 10, 3))
[2, 5, 8]
>>> list(range(5, 0, -1))
[5, 4, 3, 2, 1]
```

Example: $\sum_{i=1}^{10} i^2$

```
total = 0
for i in range(1, 11):
    total += i**2
print(total)
```

```
385
```

Python 3. `range()` does not build a list: it produces the numbers on demand. Use `list(range(5))` to see them. (In Python 2, `range` returned a list.)

Looping idioms: `enumerate()` and `zip()`

```
symbols = ["H", "C", "N", "O"]
masses = [1.008, 12.011, 14.007, 15.999]
```

Index and element at the same time

```
for i, sym in enumerate(symbols):
    print(i, sym)
```

```
0 H
1 C
2 N
3 O
```

Two sequences in parallel

```
for sym, m in zip(symbols, masses):
    print(f"{sym:2s} {m:7.3f}")
```

```
H    1.008
C   12.011
N   14.007
O   15.999
```

Tip. Prefer `for x in data:` to `for i in range(len(data)):` when you do not need the index; use `enumerate` when you need both.

Handling errors: `try` / `except`

docs ↗

An error raises an **exception**: the program stops with a traceback. `try`/`except` lets you catch it and react:

```
try:
    ...     # code that may fail
except ValueError:
    ...     # only if a ValueError occurs
```

```
text = input("Temperature (K): ")
try:
    T = float(text)
    print(f"T = {T} K")
except ValueError:
    print(f"'{text}' is not a number")
```

```
Temperature (K): abc
'abc' is not a number
```

Ask again until the input is valid

```
while True:
    try:
        T = float(input("T (K): "))
        break       # valid input
    except ValueError:
        print("Not a number, try again")
print(f"T = {T} K")
```

```
T (K): abc
Not a number, try again
T (K): 298.15
T = 298.15 K
```

Watch out! Catch *specific* exceptions: a bare `except:` also hides your own typos.

More on exceptions

The complete statement

```
try:
    ...   # may raise an exception
except ValueError:
    ...   # if ValueError
except (TypeError, KeyError):
    ...   # if any of these
except Exception as e:
    ...   # any other error, in e
else:
    ...   # if there was no error
finally:
    ...   # always, error or not
```

Only `try` and at least one `except` (or `finally`) are required.

Getting the error message

```
try:
    f = open("no.xyz")
except FileNotFoundError as e:
    print(e)
```

```
[Errno 2] No such file or directory: 'no.xyz'
```

Raising your own errors

```
T = -10.0
if T <= 0:
    raise ValueError("T must be positive")
```

```
ValueError: T must be positive
```

Stopping with a clear message is better than continuing with wrong data.

4 Sequences: lists, tuples and strings

Lists and tuples

docs ↗

Ordered collections of objects of any type (mixed types allowed)

```
a = [1, 2, 4.3, "table", True]    # list
b = (1, 2, 4.3, "table", True)    # tuple
single = (5,)        # note the comma!
```

Empty ones: `[]` and `()`. Convert with `tuple(a)` and `list(b)`.

They can be traversed with `for`

```
numbers = [1, 5, 78, 12, 45]
for number in numbers:
    print(number, end=" ")
```

```
1 5 78 12 45
```

How to store a matrix?

For example, as a list of lists (one list per row):

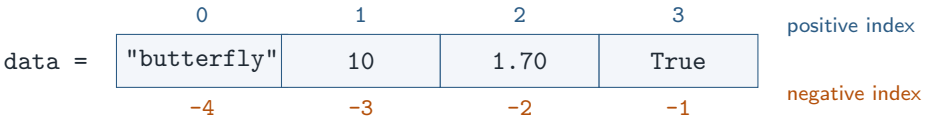
```
matrix = [[0, 1, 2],
          [1, 0, 3],
          [2, 3, -1]]
```

$$\begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 3 \\ 2 & 3 & -1 \end{pmatrix}$$

Looking ahead. For numerical work with vectors and matrices, NumPy arrays are much faster and more convenient than lists of lists.

Indexing

Elements are accessed by their position (an integer *index*), starting at 0. Negative indices count from the end.




```
>>> data = ["butterfly", 10, 1.70, True]
>>> data[0]
'butterfly'
>>> data[1], data[-3]
(10, 10)
>>> data[-1]
True
```

```
>>> data[4]
IndexError: list index out of range
>>> len(data)
4
```

The last valid index is `len(data) - 1`, the same element as `data[-1]`.

Nested lists

Elements of a list can be lists themselves; add one index per level:

```
>>> x = ["a", ["bb", ["ccc", "ddd"], "ee"]]
>>> x[0]
'a'
>>> x[1]
['bb', ['ccc', 'ddd'], 'ee']
>>> x[1][1]
['ccc', 'ddd']
>>> x[1][1][0]
'ccc'
```

Matrices: `matrix[i][j]` is the element in row `i`, column `j`

```
>>> matrix = [[0, 1, 2],
...           [1, 0, 3],
...           [2, 3, -1]]
>>> matrix[2]
[2, 3, -1]
>>> matrix[2][1]
3
```

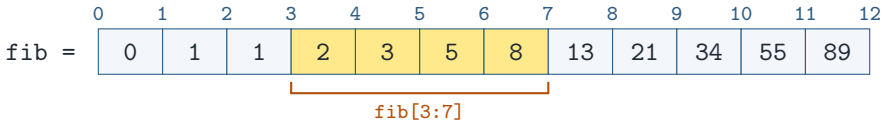
$$\begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 3 \\ 2 & 3 & -1 \end{pmatrix} \leftarrow \text{row } 2$$

$$\uparrow$$

column 1

Slicing

`seq[start:stop]` returns the elements from index `start` up to, but *not including*, index `stop`. Think of the indices as pointing *between* elements. General form: `seq[start:stop:step]`.




```
>>> fib[3:7]
[2, 3, 5, 8]
>>> fib[:4]      # from start
[0, 1, 1, 2]
>>> fib[9:]      # to the end
[34, 55, 89]
>>> fib[-3:]     # last three
[34, 55, 89]
```

```
>>> fib[::2]        # every 2nd
[0, 1, 3, 8, 21, 55]
>>> fib[::-1]       # reversed
[89, 55, 34, 21, 13, 8, 5, 3, 2, 1, 1, 0]
>>> fib[2:-2]        # drop 2 at each end
[1, 2, 3, 5, 8, 13, 21, 34]
>>> fib[10:100]     # no IndexError
[55, 89]
```

Operations on sequences

These work for lists, tuples and (most of them) strings. They create new objects and leave the original unchanged. In the examples, `v = [3, 4, 8, 9, 1, 1]`.

Operation	Example	Result
Concatenation	`[1, 3] + [2, 4]`	`[1, 3, 2, 4]`
Repetition	`(1, 2) * 3`	`(1, 2, 1, 2, 1, 2)`
Length	`len(v)`	`6`
Membership	`3 in v`	`True`
	`11 not in v`	`True`
Count occurrences	`v.count(1)`	`2`
Position of a value	`v.index(8)`	`2`
Sum, minimum, maximum	`sum(v), min(v), max(v)`	`(26, 1, 9)`
Sorted copy	`sorted(v)`	`[1, 1, 3, 4, 8, 9]`

Tip. `in` is the simplest way to check whether a value is present: `if "C" in symbols:`

Lists are mutable, tuples are not

Lists and tuples behave identically except for two things: the brackets, `[ ]` vs `( )`, and *mutability*.

List: can change

```
lst = [1, 2, 3, 4]
lst[2] = "sea"
print(lst)
```

```
[1, 2, 'sea', 4]
```

Tuple: fixed once created

```
tup = (1, 2, 3, 4)
tup[2] = "sea"
```

```
TypeError: 'tuple' object does not support item assignment
```

Why use tuples?

- Data that should not change: a point (x, y, z), quantum numbers (n, l, m_l), ...
- They can be dictionary keys (lists cannot)
- *Unpacking*: `n, l, ml = (2, 1, -1)` assigns the three values at once

List methods

docs ↗

Methods are functions attached to an object: `object.method(arguments)`. These modify the list *in place*:

Adding: `append(value)`, `insert(index, value)`

```
lst = [1, 2, 3, 4]
lst.append(8)        # at the end
print(lst)
lst.insert(1, 9)     # before index 1
print(lst)
```

```
[1, 2, 3, 4, 8]
[1, 9, 2, 3, 4, 8]
```

`lst.extend([5, 6])` appends all the elements of another sequence.

Removing: `pop(index)`, `remove(value)`

```
lst = [6, 9, 11, 3]
x = lst.pop(2)       # remove index 2
print(lst, x)
lst.remove(9)        # first 9 found
print(lst)
lst.remove(2)
```

```
[6, 9, 3] 11
[6, 3]
ValueError: list.remove(x): x not in list
```

`lst.pop()` without an index removes the last element.

Sorting, and building lists

Sorting

```
>>> lst = [30, 1, 100]
>>> sorted(lst)         # new list
[1, 30, 100]
>>> lst.sort()          # in place
>>> lst
[1, 30, 100]
>>> lst.reverse()
>>> lst
[100, 30, 1]
>>> sorted(["Na", "C", "H", "Cl"])
['C', 'Cl', 'H', 'Na']
```

Watch out! In-place methods return `None`: after `lst = lst.sort()` your list is gone!

Building a list in a loop

```
squares = []
for i in range(6):
    squares.append(i**2)
print(squares)
```

```
[0, 1, 4, 9, 16, 25]
```

The same with a list comprehension

```
squares = [i**2 for i in range(6)]
values = [3, -1, 4, -1, 5]
pos = [v for v in values if v > 0]
print(pos)
```

```
[3, 4, 5]
```

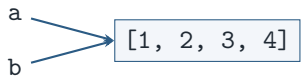
Pitfall: names are labels, not boxes

EXTRA

Assignment does *not* copy a list: both names refer to the same object.

```
a = [1, 2, 3]
b = a          # NOT a copy
b.append(4)
print(a)
```

```
[1, 2, 3, 4]
```



Tip. To get an independent copy use `b = a.copy()` (or `a[:]`, or `list(a)`).

The same trap appears when building a matrix with $*$:

```
m = [[0] * 3] * 3  # same row x3
m[0][0] = 1
print(m)
```

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

Correct: create a new row in every iteration

```
m = [[0] * 3 for i in range(3)]
m[0][0] = 1
print(m)
```

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Equality and identity: == vs is

- == compares **values**: do they contain the same?
- is compares **identity**: are both names the same object? (id() gives an object's identity number)

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> c = a
>>> a == b
True
>>> a is b          # two lists
False
>>> a is c          # one list
True
```

When to use is

Almost only to check for None:

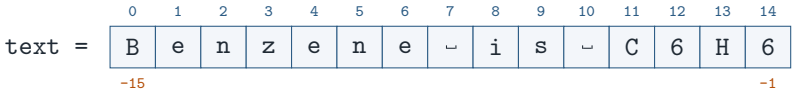
```
if result is None:
    print("not computed yet")
```

```
not computed yet
```

Watch out! Do not use is to compare numbers or strings: the result depends on implementation details. Python even warns: `SyntaxWarning: "is" with 'int' literal. Did you mean "=="?`

Rule of thumb. == for values, is for None.

Strings are sequences too



Indexing and slicing work exactly as for lists:

```
>>> text = "Benzene is C6H6"
>>> len(text)
15
>>> text[0], text[-1]
('B', '6')
>>> text[:7]
'Benzene'
>>> text[11:]
'C6H6'
>>> text[::-1]
'6H6C si enezneB'
```

Strings are *immutable*, like tuples: `text[0] = "b"` raises a `TypeError`. You can rebind the name (`text = "toluene"`), or build a new string:

```
>>> "b" + text[1:]
'benzene is C6H6'
```

Membership checks for substrings:

```
>>> "C6" in text
True
>>> "c6" in text
False
```

Useful string methods

docs ↗

String methods return a *new* string (or list):

```
>>> name = "  Water  "
>>> name.strip().upper()
'WATER'
>>> "H2O".replace("H", "D")
'D2O'
>>> "CH3CH2OH".count("C")
2
>>> "benzene".find("z")    # position
3
>>> "C, H, O".split(",")
['C', ' H', ' O']
>>> "C, H, O".split(", ")
['C', 'H', 'O']
>>> "-".join(["H", "O", "H"])
'H-O-H'
```

Example: reading a line of an XYZ file

`split()` without arguments splits on any whitespace, and is the basic tool for reading the output of quantum chemistry programs:

```
line = "O   0.0000   0.0000   0.1173\n"
fields = line.split()
print(fields)
symbol = fields[0]
z = float(fields[3])
print(symbol, z, type(z))
```

```
['O', '0.0000', '0.0000', '0.1173']
O 0.1173 <class 'float'>
```

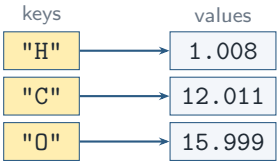
5 Dictionaries

Dictionaries

docs ↗

- Like a list, but more general: elements are found by a **key** instead of a position
- Each key points to a **value**: a key → value mapping
- Defined with braces: `{key: value, ...}`
- Keys must be unique and immutable: strings (the most common), integers, booleans, tuples (but not lists); floats are possible but not recommended
- Dictionaries keep the order in which keys were inserted

```
masses = {"H": 1.008, "C": 12.011,
          "O": 15.999}
```




```
>>> {[1, 2]: "x"}
TypeError: unhashable type: 'list'
```

Watch out! `True == 1` and `1.0 == 1`, so these keys are the *same* key.

Creating and modifying dictionaries

Start from an empty dictionary, {}, and add elements with square brackets:

```
Z = {}                  # empty dictionary
Z["H"] = 1              # key "H" -> value 1
Z["He"] = 2
Z["Li"] = 30
print(Z)
Z["Li"] = 3             # replaces 30
del Z["He"]             # removes key "He"
print(Z, len(Z))
```

```
{'H': 1, 'He': 2, 'Li': 30}
{'H': 1, 'Li': 3} 2
```

Or write all the pairs directly:

```
Z = {"H": 1, "He": 2, "Li": 3}
```

Values can be any object

```
water = {
    "formula": "H2O",
    "atoms": ["O", "H", "H"],
    "charge": 0,
    "dipole": 1.85,    # debye
}
```

Remember. {} is an empty *dictionary*, not an empty set.

Accessing values

Values are obtained with their **keys**, never with positions:

```
>>> Z = {"H": 1, "He": 2, "Li": 3}
>>> Z["He"]
2
>>> Z[0]
KeyError: 0
```

Careful with keys that do not exist

```
>>> Z["Be"]
KeyError: 'Be'
>>> Z[2]          # a value!
KeyError: 2
```

Two safe options

1. Check first with `in` (which looks at the *keys*):

```
>>> "Be" in Z
False
>>> 2 in Z
False
```

2. Use `get()`, which returns a default value instead of failing:

```
>>> print(Z.get("Be"))
None
>>> Z.get("Be", 0)
0
```

Dictionary methods and loops

`keys()`, `values()` and `items()` give the keys, the values and the (key, value) pairs:

```
>>> Z = {"H": 1, "He": 2, "Li": 3}
>>> Z.keys()
dict_keys(['H', 'He', 'Li'])
>>> list(Z.keys())
['H', 'He', 'Li']
>>> list(Z.values())
[1, 2, 3]
>>> list(Z.items())
[('H', 1), ('He', 2), ('Li', 3)]
```

Python 3. These methods return *views*, not lists (they were lists in Python 2); wrap them in `list()` if you need a list.

All three can be used in `for` loops

```
for sym in Z:           # keys
    print(sym, end=" ")
print()
for sym, n in Z.items():
    print(f"{sym:>2} -> {n}")
```

```
H He Li
 H -> 1
He -> 2
Li -> 3
```

Putting it together: molar mass

```python
masses = {"H": 1.008, "C": 12.011,
          "N": 14.007, "O": 15.999}    # g/mol

ethanol = {"C": 2, "H": 6, "O": 1}     # C2H5OH

M = 0.0
for element, count in ethanol.items():
    M += count * masses[element]

print(f"M(C2H5OH) = {M:.3f} g/mol")
```

```
M(C2H5OH) = 46.069 g/mol
```

What is used here

- two dictionaries: a *lookup table* (symbol → mass) and a composition (symbol → count)
- a `for` loop over `items()`
- an accumulator with `+=`
- an f-string to format the result

Try it. Compute the molar mass of glucose, $C_6H_{12}O_6$. What happens if the formula contains an element that is not in `masses`?

Comprehensions

List comprehensions

```
[expr for item in iterable if condition]
```

```
>>> [x**2 for x in range(1, 11)]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>> [x**2 for x in range(1, 11) if x % 2 == 0]
[4, 16, 36, 64, 100]
```

Dictionary comprehensions

```
{key: value for item in iterable if condition}
```

```
>>> {x: x**2 for x in range(1, 6)}
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
>>> symbols = ["H", "He", "Li", "Be"]
>>> {s: Z for Z, s in enumerate(symbols, start=1)}
{'H': 1, 'He': 2, 'Li': 3, 'Be': 4}
```

`enumerate(..., start=1)` counts from 1 instead of 0.

Generator expressions

With parentheses, values are produced one at a time and no list is stored. Ideal inside `sum`, `min` or `max`:

```
>>> sum(x**2 for x in range(1, 11))
385
```

The molar mass example, in one statement:

```
M = sum(n * masses[el]
        for el, n in ethanol.items())
print(f"{M:.3f}")
```

```
46.069
```

Tip. If it does not fit comfortably in one line, write a normal `for` loop.

Iterables and iterators

EXTRA

Iterables (lists, tuples, strings, dictionaries, `range`, files…) can be traversed by `for`. `iter()` gives an **iterator**, and `next()` returns its elements one by one:

```
>>> it = iter(["C", "H", "O"])
>>> next(it), next(it), next(it)
('C', 'H', 'O')
>>> next(it)              # exhausted
StopIteration
```

Looking ahead. Later we will write our own iterators: *generator functions* (with `yield`) and classes.

What a `for` loop does for you

`for atom in atoms:` is equivalent to:

```
it = iter(atoms)
while True:
    try:
        atom = next(it)
    except StopIteration:
        break
    ...   # loop body
```

Watch out! `zip`, `enumerate` and files can be used only once:

```
>>> pairs = zip(["H", "O"], [1, 16])
>>> list(pairs)
[('H', 1), ('O', 16)]
>>> list(pairs)
[]
```

6 Reading and writing files

Opening and closing files

docs ↗

So far: the **standard input** (keyboard, `input()`) and the **standard output** (screen, `print()`). Files keep data between runs and let programs talk to each other, e.g. reading the output of a quantum chemistry code.

The basic pattern

```
f = open("myfile.txt", "r")     # open
...                             # use f
f.close()                       # close
```

Better: the `with` statement

```
with open("myfile.txt", "r") as f:
    ...                         # use f
# here the file is already closed
```

`with` closes the file for you, even if an error occurs.

Modes (second argument of `open`)

Mode	Meaning
`"r"`	read (the default)
`"w"`	write: creates the file, or *erases* it
`"a"`	append: write at the end
`"r+"`	read and write
`"t"`, `"b"`	text (default) or binary, e.g. `"rb"`

Watch out! `"w"` empties an existing file without asking. `"r"` on a missing file raises a `FileNotFoundError` (catch it with `try`/`except`).

Tip. Paths are relative to the folder where you run the script.

Writing a file

Write the geometry of water in XYZ format:

```
atoms = [["O", 0.0,  0.0,     0.1173],
         ["H", 0.0,  0.7572, -0.4692],
         ["H", 0.0, -0.7572, -0.4692]]

with open("water.xyz", "w") as f:
    f.write(f"{len(atoms)}\n")
    f.write("water molecule\n")
    for s, x, y, z in atoms:
        f.write(f"{s} {x:9.4f} {y:9.4f} {z:9.4f}\n")
```

water.xyz

```
3
water molecule
O    0.0000    0.0000    0.1173
H    0.0000    0.7572   -0.4692
H    0.0000   -0.7572   -0.4692
```

`f.write(text)`

- takes *one string*: convert numbers first (f-strings are ideal)
- does *not* add a new line: write `"\n"` yourself

Alternative: `print` to a file

```
print(s, x, y, z, file=f)
```

The default is `file=sys.stdout`: the screen.

Minimal example

```
f = open("test.txt", "w")
f.write("hello world!\n")
f.close()
```

Reading a file

Read back the file written on the previous slide (mode `"r"` is the default):

```
with open("water.xyz") as f:
    n_atoms = int(f.readline())
    title = f.readline().strip()
    lines = f.readlines()     # the rest
print(n_atoms, title)
print(lines[0])
print(lines[0].split())
```

```
3 water molecule
O    0.0000    0.0000    0.1173

['O', '0.0000', '0.0000', '0.1173']
```

The empty line: `lines[0]` still ends in `"\n"`, and `print` adds another one.

Reading methods

`f.readline()`	next line (a string)
`f.readlines()`	list of the remaining lines
`f.read()`	the whole file, one string
`for line in f:`	one line per iteration

Watch out! Lines are read *with* their final `"\n"` and always as strings: use `strip()`, `split()`, `int()` and `float()`.

Tip. `for line in f:` reads one line at a time, so it also works for huge output files.

Putting it together: parsing an XYZ file

```
symbols = []
coords = []
with open("water.xyz") as f:
    f.readline()           # skip the
    f.readline()           # two header lines
    for line in f:
        fields = line.split()
        symbols.append(fields[0])
        xyz = [float(v) for v in fields[1:]]
        coords.append(xyz)

print(symbols)
print(coords[1])
```

```
['O', 'H', 'H']
[0.0, 0.7572, -0.4692]
```

What is used here

- `with` and `open` to read the file
- `readline()` to skip the header
- a `for` loop over the lines
- `split()` and `float()` to turn text into numbers
- lists built with append and a list comprehension

Looking ahead. For common formats, libraries do this for you: `numpy.loadtxt` (arrays), `ase.io.read` (structures), `pandas.read_csv` (tables).

7 Wrap-up and practice

Collections at a glance

	list	**tuple**	**str**	**dict**
Syntax	`[1, 2, 3]`	`(1, 2, 3)`	`"abc"`	`{"a": 1, "b": 2}`
Ordered	✓	✓	✓	✓ (insertion order)
Mutable	✓	×	×	✓
Accessed by	position	position	position	key
Slicing	✓	✓	✓	×
`len`, `in`, `for`	✓	✓	✓	✓ (on the keys)
Typical use	data that grows or changes	fixed records, coordinates	text	lookup tables: symbol → mass

One more. Python also has *sets*, unordered collections without repeated elements: `{"C", "H", "O"}`.

Python 2 vs Python 3: recognising old code

EXTRA

Much scientific code found online (and in some research groups) is still Python 2, which reached its end of life on 1 January 2020. Recognise it, and do not copy it blindly:

	Python 2 (obsolete)	**Python 3**
Printing	`print "E =", E`	`print("E =", E)`
Division	`7 / 2` → `3`	`7 / 2` → `3.5` (`7 // 2` → `3`)
Reading text	`raw_input()`	`input()` (always a string)
`range(5)`	a list	a lazy range: `list(range(5))`
`d.keys()`	a list	a view: `list(d.keys())`
Rounding	`round(2.5)` → `3.0`	`round(2.5)` → `2`
Formatting	`"%.3f" % x`	`f"{x:.3f}"` (% still works)
Text	bytes by default	Unicode by default

Tip. The quickest giveaway is `print` without parentheses.

Practice

1. **Ideal gas.** Ask for n (mol), T (K) and V (L) and print the pressure in bar and in atm with 3 decimals.
   ($R = 0.083145$ L bar mol^{-1} K^{-1}; 1 atm = 1.01325 bar)

2. **States of water.** Ask for a temperature in °C, convert it to K and print whether water is solid, liquid or gas at 1 atm. If the input is not a number, ask again.

3. **Fibonacci.** With a `while` loop, build the list of all Fibonacci numbers smaller than 1000. How many are there?

4. **Molar mass.** Extend the molar-mass example to glucose, $C_6H_{12}O_6$ (expected: 180.156 g/mol).

5. **Bond lengths.** Store the geometry of water (Å) as a list of lists and compute the two O–H distances with `math.sqrt`:

   ```
   O   0.0000   0.0000   0.1173
   H   0.0000   0.7572  -0.4692
   H   0.0000  -0.7572  -0.4692
   ```

   (expected: 0.958 Å)

6. **Files.** Write a file with two columns, T and $k(T)$ from the Arrhenius example, for $T = 250, 300, \ldots, 500$ K. Then read it back and print the largest k.

Further reading

- **The Python Tutorial** (official): chapters 3–5, 7 and 8 cover this unit
  `docs.python.org/3/tutorial/`
- **Built-in types**: numbers, sequences, strings, dictionaries
  `docs.python.org/3/library/stdtypes.html`
- **The `math` module**
  `docs.python.org/3/library/math.html`
- **Input and output**: f-strings and files
  `docs.python.org/3/tutorial/inputoutput.html`
- **Format specification** (f-strings)
  `docs.python.org/3/library/string.html#formatspec`
- **PEP 8**: the style guide for Python code
  `peps.python.org/pep-0008/`
- **Real Python**: conditional statements in Python
  `realpython.com/python-conditional-statements/`